

Exercise 2 Date: 11/03/2026
Reg no: 240701553

Name: Tanisha.S

Develop and compare CLI, GUI, and Voice User Interfaces (VUI) for the same task and assess user satisfaction using Python (Tkinter for GUI, Speech Recognition for VUI), Terminal

AIM:

The aim is to develop and compare Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) for the same task, and assess user satisfaction using Python (with Tkinter for GUI and Speech Recognition for VUI) and Terminal.

PROCEDURE:

i) CLI (Command Line Interface)

CLI implementation where users can add, view, and remove tasks using the terminal.

```
tasks = []

def add_task(task):
    tasks.append(task)
    print(f"Task '{task}' added.")

def view_tasks():
    if tasks:
        print("Your tasks:")
        for idx, task in enumerate(tasks, 1):
            print(f"{idx}. {task}")
    else:
        print("No tasks to show.")

def remove_task(task_number):
    if 0 < task_number <= len(tasks):
```

```

        removed_task = tasks.pop(task_number - 1)
        print(f"Task '{removed_task}' removed.")
    else:
        print("Invalid task number.")

def main():
    while True:
        print("\nOptions: 1.Add Task  2.View Tasks  3.Remove Task  4.Exit")
        choice = input("Enter your choice: ")

        match choice:

            case '1':
                task = input("Enter task: ")
                add_task(task)

            case '2':
                view_tasks()

            case '3':
                task_number = int(input("Enter task number to remove: "))
                remove_task(task_number)

            case '4':
                print("Exiting...")
                break

            case _:
                print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()

```

Sample Output:

```

PS C:\Users\tanis\OneDrive\Documents\UI> & "C:/Program Files/Python313/python3.13t.exe" c:/Users/tanis/OneDrive/
Documents/UI/CLI
Options: 1.Add Task  2.View Tasks  3.Remove Task  4.Exit
Enter your choice: 1
Enter task: Do assignment
Task 'Do assignment' added.

Options: 1.Add Task  2.View Tasks  3.Remove Task  4.Exit
Enter your choice: 1
Enter task: buy groceries
Task 'buy groceries' added.

Options: 1.Add Task  2.View Tasks  3.Remove Task  4.Exit
Enter your choice: 2
Your tasks:
1. Do assignment
2. buy groceries

Options: 1.Add Task  2.View Tasks  3.Remove Task  4.Exit
Enter your choice: 3
Enter task number to remove: 2
Task 'buy groceries' removed.

Options: 1.Add Task  2.View Tasks  3.Remove Task  4.Exit
Enter your choice: 4
Exiting...

```

ii) GUI (Graphical User Interface)

Tkinter to create a simple GUI for our To-Do List application.

```

import tkinter as tk

from tkinter import messagebox

tasks = []

def task_action(action):

    match action:

        case "add":

            task = task_entry.get()

```

```
        if task:

            tasks.append(task)

            task_entry.delete(0, tk.END)

            update_task_list()

        else:

            messagebox.showwarning("Warning", "Task cannot be empty")

    case "remove":

        selected_task_index = task_list.curselection()

        if selected_task_index:

            tasks.pop(selected_task_index[0])

            update_task_list()

    case _:

        messagebox.showerror("Error", "Invalid Action")

def update_task_list():

    task_list.delete(0, tk.END)

    for task in tasks:
```

```
        task_list.insert(tk.END, task)

app = tk.Tk()

app.title("To-Do List")


task_entry = tk.Entry(app, width=40)

task_entry.pack(pady=10)


add_button = tk.Button(app, text="Add Task",

                        command=lambda: task_action("add"))

add_button.pack(pady=5)


remove_button = tk.Button(app, text="Remove Task",

                           command=lambda: task_action("remove"))

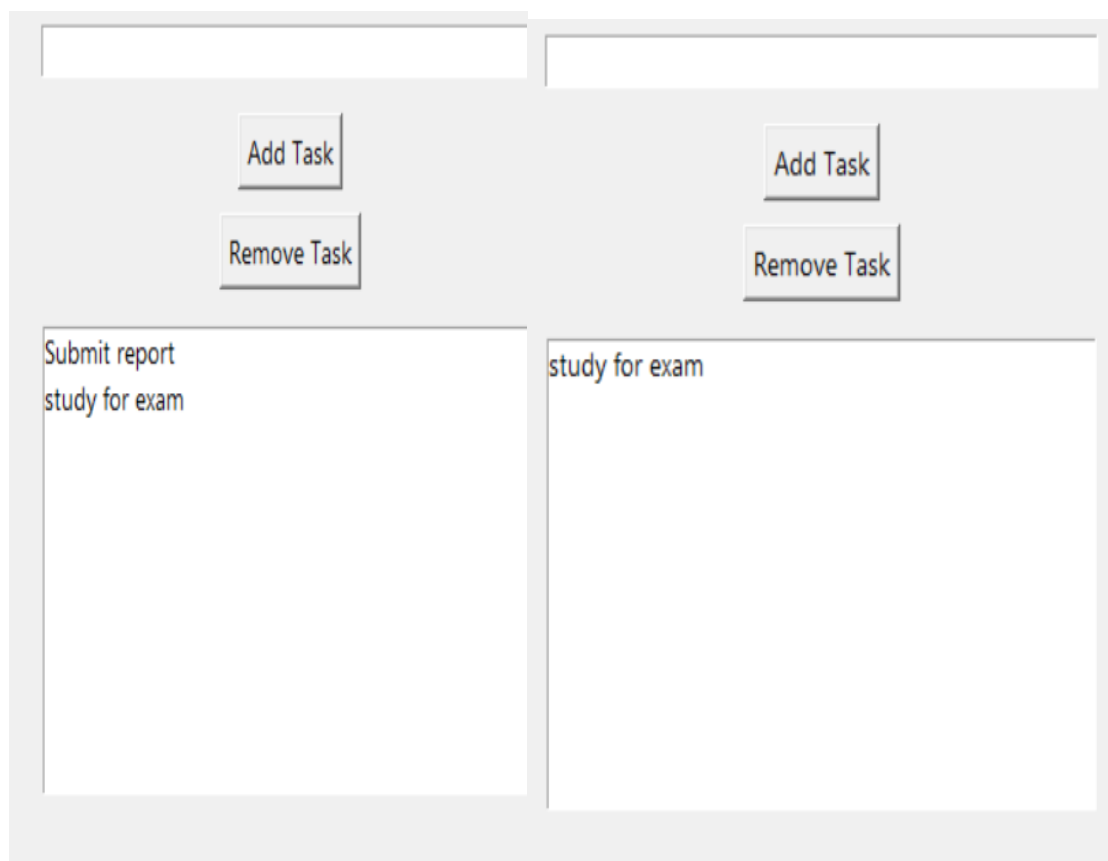
remove_button.pack(pady=5)


task_list = tk.Listbox(app, width=40, height=10)

task_list.pack(pady=10)
```

```
app.mainloop()
```

Sample Output:



iii) VUI (Voice User Interface)

speech_recognition library for voice input and the pyttsx3 library for text-to-speech output. Make sure you have these libraries installed (pip install SpeechRecognition pyttsx3).

```
import speech_recognition as sr
import pyttsx3

tasks = []

recognizer = sr.Recognizer()
engine = pyttsx3.init()

def speak(text):
    print("Assistant:", text)
    engine.say(text)
    engine.runAndWait()

def add_task(task):
    tasks.append(task)
    speak(f"Task {task} added")

def view_tasks():
    if tasks:
        speak("Your tasks are")
        for i, task in enumerate(tasks, start=1):
            speak(f"Task {i} {task}")
    else:
        speak("No tasks to show")

def remove_task(task_number):
    if 0 < task_number <= len(tasks):
        removed_task = tasks.pop(task_number - 1)
        speak(f"Task {removed_task} removed")
    else:
        speak("Invalid task number")

def recognize_speech():
    with sr.Microphone() as source:
        print("Listening...")
```

```

        recognizer.adjust_for_ambient_noise(source,
duration=1)

        try:
            audio = recognizer.listen(source, timeout=5,
phrase_time_limit=5)
            except sr.WaitTimeoutError:
                print("No speech detected")
                return None

        try:
            command = recognizer.recognize_google(audio)
            print("You said:", command)
            return command.lower()

        except sr.UnknownValueError:
            speak("Sorry, I did not understand that")
            return None

        except sr.RequestError:
            speak("Speech service unavailable")
            return None

def main():

    speak("Voice task manager started")

    while True:

        speak("Options are add, view, remove, or exit")

        command = recognize_speech()

        if not command:
            continue

        if "add" in command:
            speak("What is the task?")
            task = recognize_speech()
            if task:

```



```
        add_task(task)

    elif "view" in command:
        view_tasks()

    elif "remove" in command:
        speak("Which task number to remove?")
        number = recognize_speech()

        try:
            task_number = int(number)
            remove_task(task_number)
        except:
            speak("Please say a valid number")

    elif "exit" in command:
        speak("Exiting the program")
        break

    else:
        speak("Invalid option")

if __name__ == "__main__":
    main()
```

Sample Output:

